arXiv:2502.17521v2 [cs.LG] 29 Sep 2025

Benchmarking Large Language Models Under Data Contamination: A Survey from Static to Dynamic Evaluation

Simin Chen[1] Yiming Chen[2*] Zexin Li[3*] Yifan Jiang[4] Zhongwei Wan[5] Yixin He[4]
Dezhi Ran[6] Tianle Gu[7] Haizhou Li[2,8] Tao Xie[6] Baishakhi Ray[1]

[1]Columbia University [2]National University of Singapore [3]University of California, Riverside
[4]University of Southern California [5]The Ohio State University [6]Peking University
[7]Tsinghua University [8]The Chinese University of Hong Kong, Shenzhen

Static-to-Dynamic-LLMEval GitHub Repository

Abstract

In the era of evaluating large language models (LLMs), data contamination has become an increasingly prominent concern. To address this data contamination risk, LLM benchmarking has evolved from a *static* to a *dynamic* paradigm. In this work, we conduct an in-depth analysis of existing *static* and *dynamic* benchmarks for evaluating LLMs. We first examine methods that enhance *static* benchmarks and identify their inherent limitations. We then highlight a critical gap—the lack of standardized criteria for evaluating *dynamic* benchmarks. Based on this observation, we propose a series of optimal design principles for *dynamic* benchmarking and analyze the limitations of existing *dynamic* benchmarks. This survey provides a concise yet comprehensive overview of recent advancements in data contamination research, offering valuable insights and a clear guide for future research efforts. We maintain a GitHub repository to continuously collect both static and dynamic benchmarks for LLMs.

1 Introduction

The field of natural language processing (NLP) has advanced rapidly in recent years, driven by breakthroughs in Large Language Models (LLMs) such as GPT-4, Claude3, and DeepSeek (Achiam et al., 2023; Liu et al., 2024; Wan et al., 2023). Trained on vast amounts of Internet-sourced data, these models have demonstrated remarkable capabilities across various applications, including code generation, text summarization, computer use, and mathematical reasoning (Codeforces, 2025; Ran et al., 2024; Hu et al., 2024).

To develop and improve LLMs, beyond advancements in model architectures and training algorithms, a crucial area of research focuses on effectively evaluating their intelligence. Traditionally, LLM evaluation has relied on *static* benchmarking, which involves using carefully curated human-crafted datasets and assessing model performance with appropriate metrics (Wang et al., 2018; Achiam et al., 2023; Gunasekar et al., 2023; Ran et al., 2025).

*Corresponding authors: yiming.chen@u.nus.edu, zli536@ucr.edu

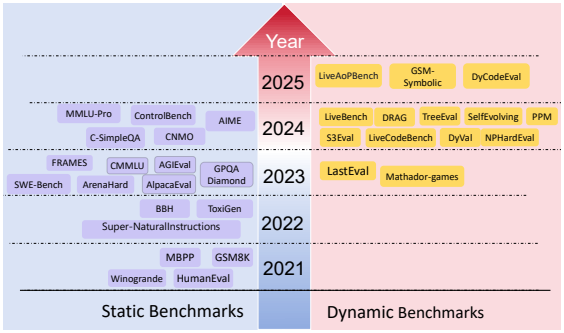

Figure 1: The progress of benchmarking LLMs.

However, because these *static* benchmarks are released on the Internet for transparent evaluation, and LLMs gather as much data as possible from the Internet for training, potential data contamination is unavoidable (Magar and Schwartz, 2022; Deng et al., 2024b; Li et al., 2024d; Balloccu et al., 2024). Data contamination occurs when benchmark data is inadvertently included in the training phase of LLMs, leading to inflated and misleading performance assessments. Although this issue has long been recognized—rooted in the fundamental machine learning principle of separating training and test sets—it has become more critical with the rise of LLMs, which often scrape vast amounts of publicly available Internet data (Achiam et al., 2023), increasing the risk of contamination. Furthermore, due to privacy and commercial concerns, tracing the exact training data of these models is challenging—if not impossible—complicating efforts to detect and mitigate potential contamination.

To mitigate the risk of data contamination in LLM benchmarking, researchers have proposed

several enhancements to static evaluation methods, including data encryption (Jacovi et al., 2023) and post-hoc contamination detection (Shi et al., 2024). However, due to the inherent limitations of static methods—such as unverifiable data exposure—these enhancements have seen limited adoption. As a result, researchers have shifted toward new *dynamic* benchmarking paradigms, as illustrated in Fig. 1. Dynamic methods aim to reduce contamination risk either by continuously updating benchmark datasets based on LLM training timestamps (White et al., 2024; Jain et al., 2024), or by regenerating test data to reconstruct and replace original benchmarks (Chen et al., 2024a; Zhou et al., 2025; Mirzadeh et al., 2025).

Although many dynamic benchmarking methods have been proposed to promote fair and transparent evaluation of LLMs, most existing work primarily highlights the advantages of these dynamic benchmarks (White et al., 2024). However, the question remains: *what are the potential trade-offs of using dynamic benchmarks to evaluate LLMs?* The limitations of dynamic benchmarking—such as the computational overhead of continuous updates, and the need for reliable timestamp metadata—are not yet fully explored.

Moreover, existing surveys on LLM data contamination have mainly focused on post-hoc detection methods (Deng et al., 2024a; Ravaut et al., 2024; Xu et al., 2024a; Dong et al., 2024; Balloccu et al., 2024), offering little attention to the emerging landscape of dynamic benchmarking strategies. Considering the growing importance and adoption of dynamic benchmarking methods, it is essential to assess their effectiveness and limitations. Unfortunately, our empirical survey of existing dynamic benchmarking methods reveals that their evaluations are highly fragmented. To date, there is no systematic work that defines clear evaluation criteria for dynamic benchmarks themselves. Moreover, existing reviews often overlook a detailed comparison of the strengths and weaknesses of different dynamic methods, leaving a gap in understanding their practical trade-offs and applicability.

To bridge this gap, we first conduct a systematic survey of benchmarking methods for LLMs designed to mitigate the risk of data contamination, covering both *static* and *dynamic* benchmarks. We summarize state-of-the-art methods and provide an in-depth discussion of their strengths and limitations. Furthermore, we are the first to summarize and abstract a set of criteria for evaluating *dynamic* benchmarks. Our study reveals that existing *dynamic* benchmarks do not fully satisfy these proposed criteria, implying the imperfection of current design. We hope that our criteria will provide valuable insights for the future design and standardization of *dynamic* benchmarking methods.

The paper is organized as shown in Fig. 2. We first review the background on data contamination (§2), and then survey *static* benchmarks and their enhancements for mitigating data contamination (§3). Next, we introduce key principles and existing methods for *dynamic* benchmarking (§4). Finally, we discuss open challenges and future directions (§5).

2 Background

2.1 Data Contamination

Data contamination arises when LLM training data $\mathcal{D}_{\text{train}}$ improperly overlaps with evaluation data $\mathcal{D}_{\text{test}}$, undermining performance validity. We review existing work and formalize the definition.

Exact Contamination. Exact contamination occurs when there is any exact duplicate in the benchmark dataset

$$\exists\, d \quad \text{s.t.} \quad d \in \mathcal{D}_{\text{train}} \text{ and } d \in \mathcal{D}_{\text{test}}$$

In other words, there exists a data point d that is in both $\mathcal{D}_{\text{train}}$ and $\mathcal{D}_{\text{test}}$. Common cases include verbatim test examples appearing in training corpora, code snippets from benchmark implementations, or documentation leaks.

Syntactic Contamination. Syntactic contamination occurs when a test data point could be found in the training dataset after a syntactic transformation, such that

$$\exists\, d \quad \text{s.t.} \quad \mathcal{F}_{\text{syntactic}}(d) \in \mathcal{D}_{\text{train}} \text{ and } d \in \mathcal{D}_{\text{test}}$$

where $\mathcal{F}_{\text{syntactic}}$ denotes syntactic transformations like punctuation normalization, whitespace modification, synonym substitution, morphological variations, or syntactic paraphrasing while preserving lexical meaning.

Examples of Each Contamination. We provide contamination examples in Table 1. Syntactic contamination occurs when test data is rephrased from training data using a prefix. Whether such syntactic contamination constitutes true contamination is debated, as it is difficult to separate memorization from reasoning. In this work, we treat such transformations as contamination, since some NLP tasks rely heavily on syntax.

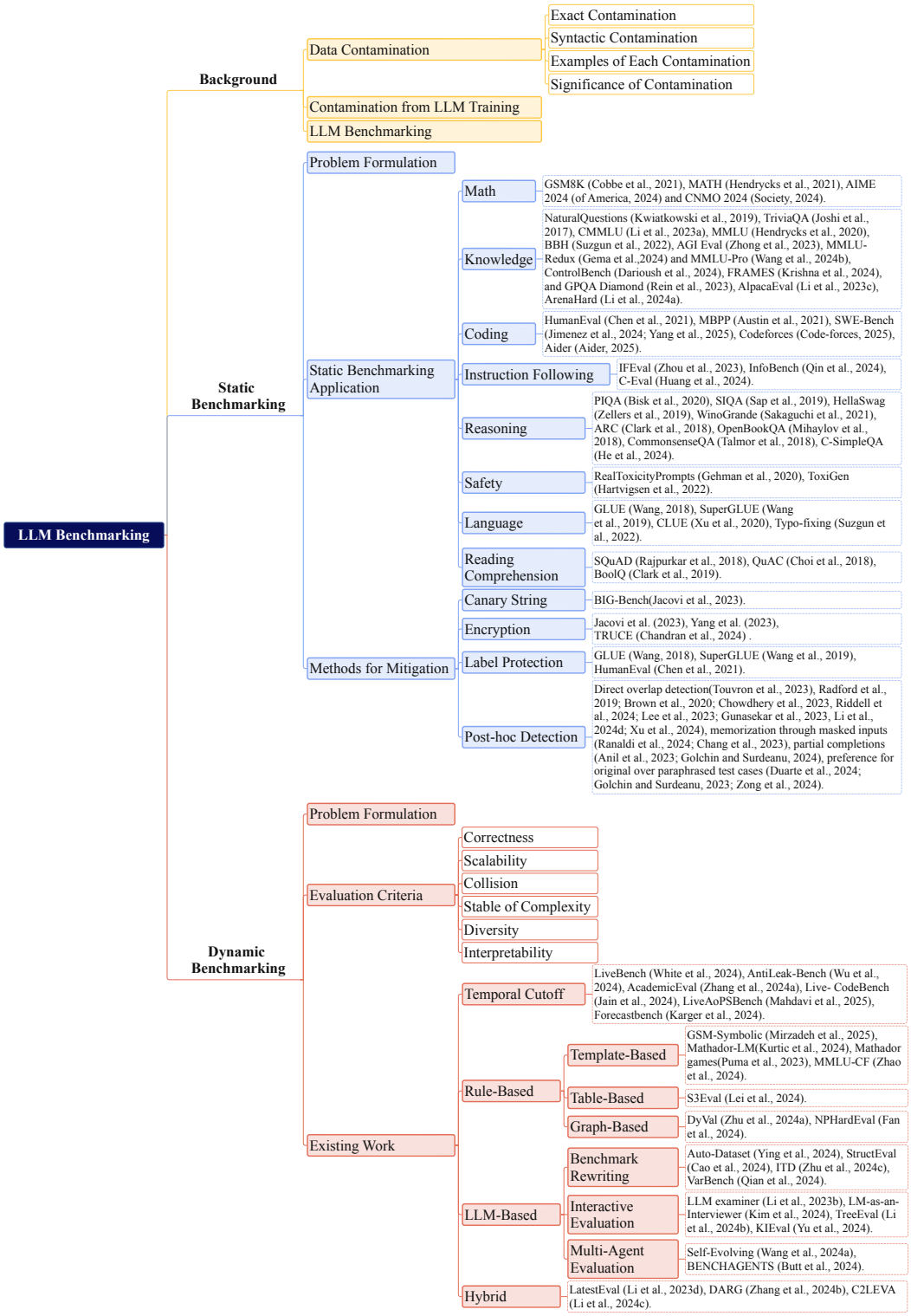

Figure 2: Taxonomy of research on benchmarking LLMs.

Contamination Type	Training Data	Testing Data
Exact Contamination	`Write a Python function to check if a number is prime.`	`Write a Python function to check if a number is prime.`
Syntactic Contamination	`Write a Python function to check if a number is prime.`	`You are a helpful code assistant for Python. Write a Python function to check if a number is prime.`

Table 1: Examples of data contamination in LLMs.